13

© 한상훈 NestJS 백엔드 마스터

NestJS 백엔드 마스터

PART 2

모듈: 경계를 긋는 단위

제2부 NestJS 핵심 · 4장 구성 요소

기능 단위로 나누고 보이는 범위를 정하는 단위
클래스는 다 있는데 Nest가 못 찾는 이유



© 한상훈 NestJS 백엔드 마스터

회차 등록 규칙

→ 회차(performance): 공연 하나가 실제로 열리는 날짜와 시각

→ 회차 등록 전: 공연 존재 확인

→ `PerformancesService` 생성자에 `ConcertsService`

예: 공연

가을 콘서트

회차

11월 14일 19시 회차

회차

11월 15일 18시 회차

© 한상훈 NestJS 백엔드 마스터

부팅 중에 멈춘 서버

터미널

```
$ node dist/apps/ticketing/main.js
LOG [NestFactory] Starting Nest application...
ERROR [ExceptionHandler] UnknownDependenciesException [Error]:
Nest can't resolve dependencies of the PerformancesService (?).
Please make sure that the argument ConcertsService at index [0]
is available in the PerformancesModule module.

Potential solutions:
- Is PerformancesModule a valid NestJS module?
- If ConcertsService is a provider, is it part of the current PerformancesModule?
- If ConcertsService is exported from a separate @Module, is that module
  imported within PerformancesModule?
```

→ 클래스 있음, import 줄 있음, 빌드 통과

→ 에러 끝: 공식 문서 '자주 보는 에러'(common errors) 페이지 주소

데이터 블록: Nest 12.2.0 에러 문구, 2026-10-26 기준, 촬영 직전 확인

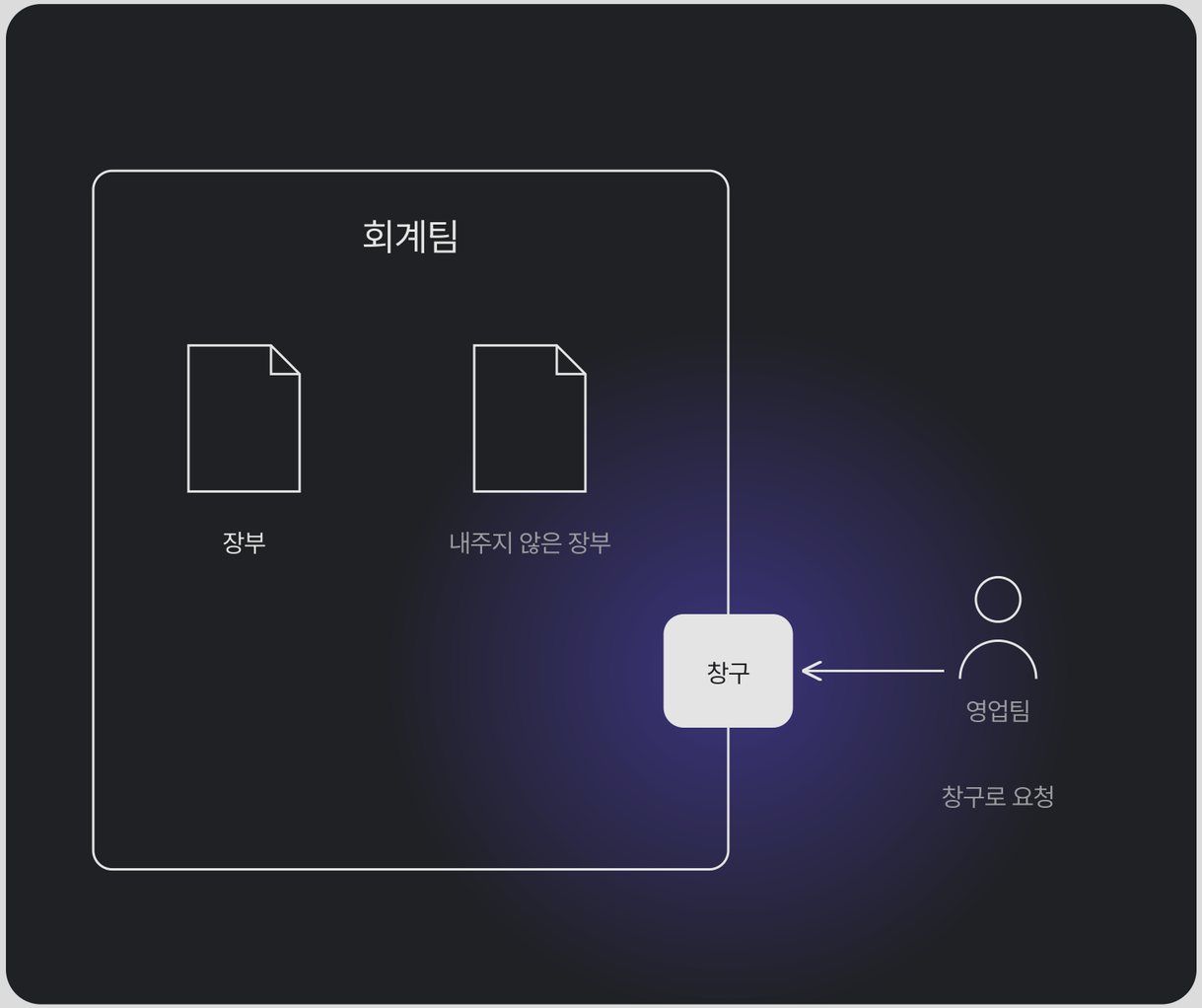
© 한상훈 NestJS 백엔드 마스터

회사 부서와 창구

→ 회계팀 장부: 회계팀 안

→ 영업팀: 장부 직접 열람 불가, 회계팀 창구로 요청

→ 창구로 내주지 않은 장부: 다른 팀은 존재도 모름



© 한상훈 NestJS 백엔드 마스터

@Module의 네 칸

칸	뜻	부서로 치면
`providers`	이 모듈이 직접 만들어 관리하는 클래스	팀 안의 장부와 직원
`controllers`	이 모듈에서 요청을 받는 클래스	민원 접수대
`imports`	가져다 쓸 다른 모듈	이용하는 다른 팀 창구
`exports`	providers 중 밖에 내주는 것	우리 팀 창구

© 한상훈 NestJS 백엔드 마스터

지금의 ConcertsModule

concerts.module.ts

```ts
import { Module } from '@nestjs/common';
import { ConcertsService } from './concerts.service.js';
import { ConcertsController } from './concerts.controller.js';
import { HoldsController } from './holds.controller.js';
import { FilesController } from '../files/files.controller.js';

@Module({
  controllers: [ConcertsController, HoldsController, FilesController],
  providers: [ConcertsService],
})
export class ConcertsModule {}
```

→ controllers: 공연, 좌석 선점 껍데기, 포스터·판매 내역 파일

→ imports 없음, exports 없음

→ ConcertsService: 이 모듈 안에서만 사용

© 한상훈 NestJS 백엔드 마스터

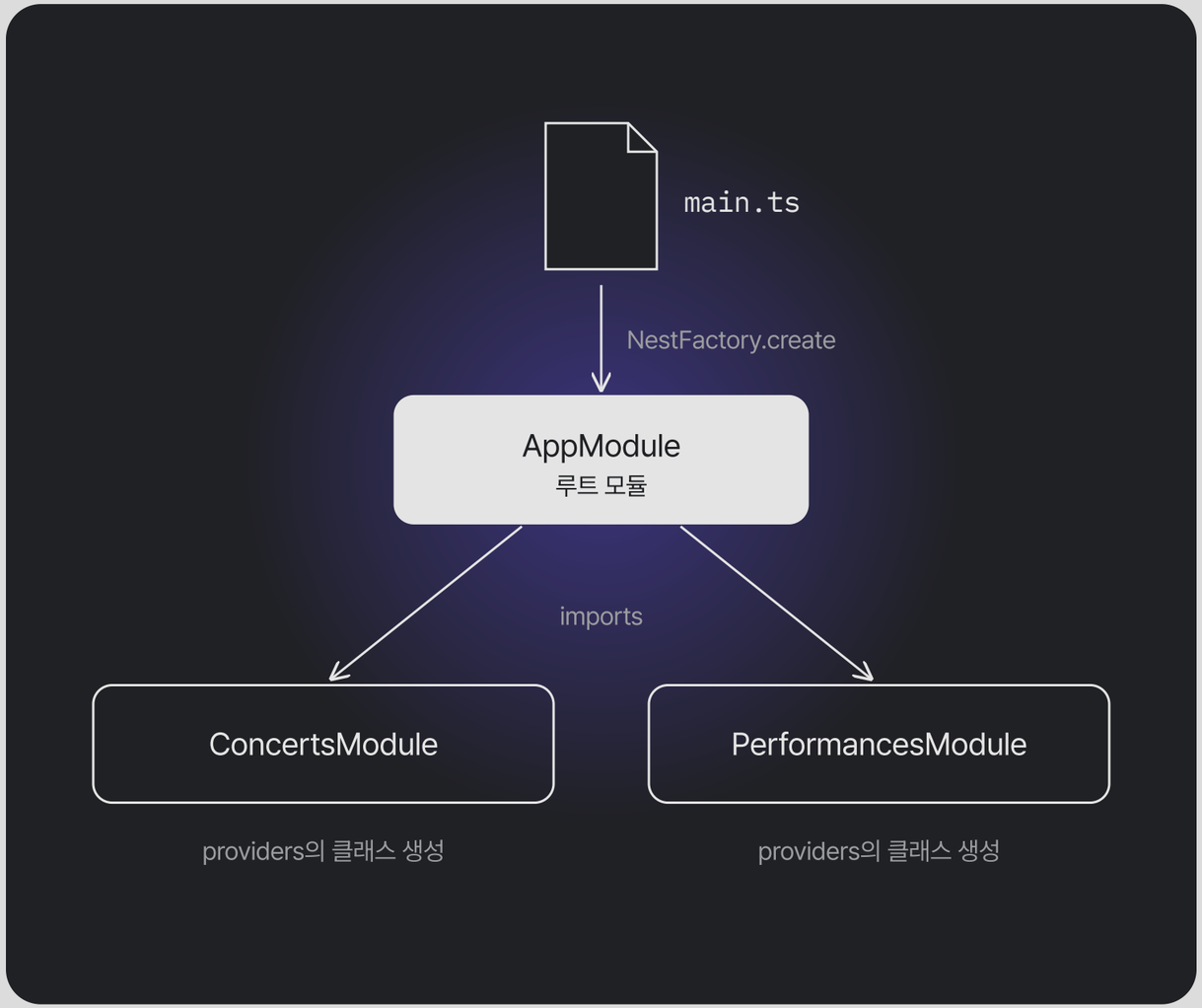
루트 모듈과 모듈 그래프

→ 루트 모듈: `main.ts`가 `NestFactory.create`에 넘긴 `AppModule`

→ Nest: 루트 모듈에서 시작, imports를 따라 모든 모듈 방문

→ 모듈 그래프: 모듈끼리 imports로 이어진 그림

→ 방문한 모듈마다 providers의 클래스 생성



© 한상훈 NestJS 백엔드 마스터

모듈마다 한 줄

부팅 로그

```
LOG [InstanceLoader] AppModule dependencies initialized +5ms
LOG [InstanceLoader] ConcertsModule dependencies initialized +0ms
LOG [InstanceLoader] PerformancesModule dependencies initialized +0ms
```

→ 그 모듈의 프로바이더 생성 완료 표시

→ 새 모듈이 이 목록에 없음: 어디에서도 imports 안 함

→ 기능 하나에 모듈 하나: 공연, 회차, 좌석

© 한상훈 NestJS 백엔드 마스터

nest g resource performances

터미널

```
$ nest g resource performances --project ticketing --type rest --crud --no-spec
CREATE apps/ticketing/src/performances/performances.controller.ts (1084 bytes)
CREATE apps/ticketing/src/performances/performances.module.ts (312 bytes)
CREATE apps/ticketing/src/performances/performances.service.ts (739 bytes)
CREATE apps/ticketing/src/performances/dto/create-performance.dto.ts (38 bytes)
CREATE apps/ticketing/src/performances/dto/update-performance.dto.ts (204 bytes)
CREATE apps/ticketing/src/performances/entities/performance.entity.ts (29 bytes)
UPDATE apps/ticketing/src/app.module.ts (439 bytes)
```

→ 모노레포: `--project`로 서비스 지정

→ AppModule imports에 PerformancesModule 자동 추가

© 한상훈 NestJS 백엔드 마스터

공연 서비스의 임시 저장소

```
@Injectable()
export class ConcertsService {
  private readonly concerts: Concert[] = [];
  private nextId = 1;

  constructor() {
    console.log('ConcertsService 생성');
  }

  create(createConcertDto: CreateConcertDto) {
    const concert = { id: this.nextId++, title: createConcertDto.title };
    this.concerts.push(concert);
    return concert;
  }

  findOne(id: number) {
    const concert = this.concerts.find((c) => c.id === id);
    if (!concert) {
      throw new NotFoundException(`공연 ${id}번이 없습니다`);
    }
    return concert;
  }
}
```

→ DB 연결 전: 서비스 안의 배열 (코드에 넣어 둔 '10월 콘서트' 대신 등록한 공연)

→ 생성자 로그: 인스턴스가 몇 개 생기는지 확인용

© 한상훈 NestJS 백엔드 마스터

공연부터 확인하는 회차 서비스

performances.service.ts

```ts
@Injectable()
export class PerformancesService {
  private readonly performances: Performance[] = [];
  private nextId = 1;

  constructor(private readonly concertsService: ConcertsService) {}

  create(createPerformanceDto: CreatePerformanceDto) {
    const { concertId, startsAt } = createPerformanceDto;
    this.concertsService.findOne(concertId); // 없으면 NotFoundException
    const performance = { id: this.nextId++, concertId, startsAt };
    this.performances.push(performance);
    return performance;
  }
}
```

→ Performance 엔티티: `id`, `concertId`, `startsAt`

→ `NotFoundException`: Nest 내장 예외, 던지면 404 응답

© 한상훈 NestJS 백엔드 마스터

에러 메시지 읽기

→ `PerformancesService (?)`: 생성자 인자 중 못 찾은 것이 물음표

→ `at index [0]`: 생성자의 첫 번째 인자

→ `available in the PerformancesModule`: 찾아본 범위는 이 모듈 하나

→ 인자가 둘이면: `(ConcertsService, ?)`처럼 찾은 것은 이름, 못 찾은 것은 물음표

© 한상훈 NestJS 백엔드 마스터

providers에 한 줄 더

performances.module.ts

```ts
import { ConcertsService } from '../concerts/concerts.service.js';

@Module({
  controllers: [PerformancesController],
  providers: [PerformancesService, ConcertsService],
})
export class PerformancesModule {}
```

→ 에러 문구의 제안: "is it part of the current PerformancesModule?"

→ 결과: 부팅 성공

© 한상훈 NestJS 백엔드 마스터

두 번 찍힌 생성 로그

부팅 로그

```
LOG [NestFactory] Starting Nest application...
ConcertsService 생성
ConcertsService 생성
LOG [InstanceLoader] AppModule dependencies initialized +28ms
LOG [InstanceLoader] PerformancesModule dependencies initialized +0ms
LOG [InstanceLoader] ConcertsModule dependencies initialized +0ms
...
LOG [NestApplication] Nest application successfully started +1ms
```

© 한상훈 NestJS 백엔드 마스터

등록한 공연이 없다는 회차 API

터미널

```
$ curl -X POST localhost:4013/concerts -d '{"title":"가을 콘서트"}'
{"id":1,"title":"가을 콘서트"}

$ curl localhost:4013/concerts
[{"id":1,"title":"가을 콘서트"}]

$ curl -X POST localhost:4013/performances -d '{"concertId":1,"startsAt":"2026-11-14T19:00"}'
{"type":"about:blank","title":"Not Found","status":404,"detail":"공연 1번이 없습니다","instance":"/performances"}
```

→ 요청 헤더(Content-Type: application/json) 생략 표기 (부록)

→ 에러 본문: 앞서 건 Problem Details 필터의 모양

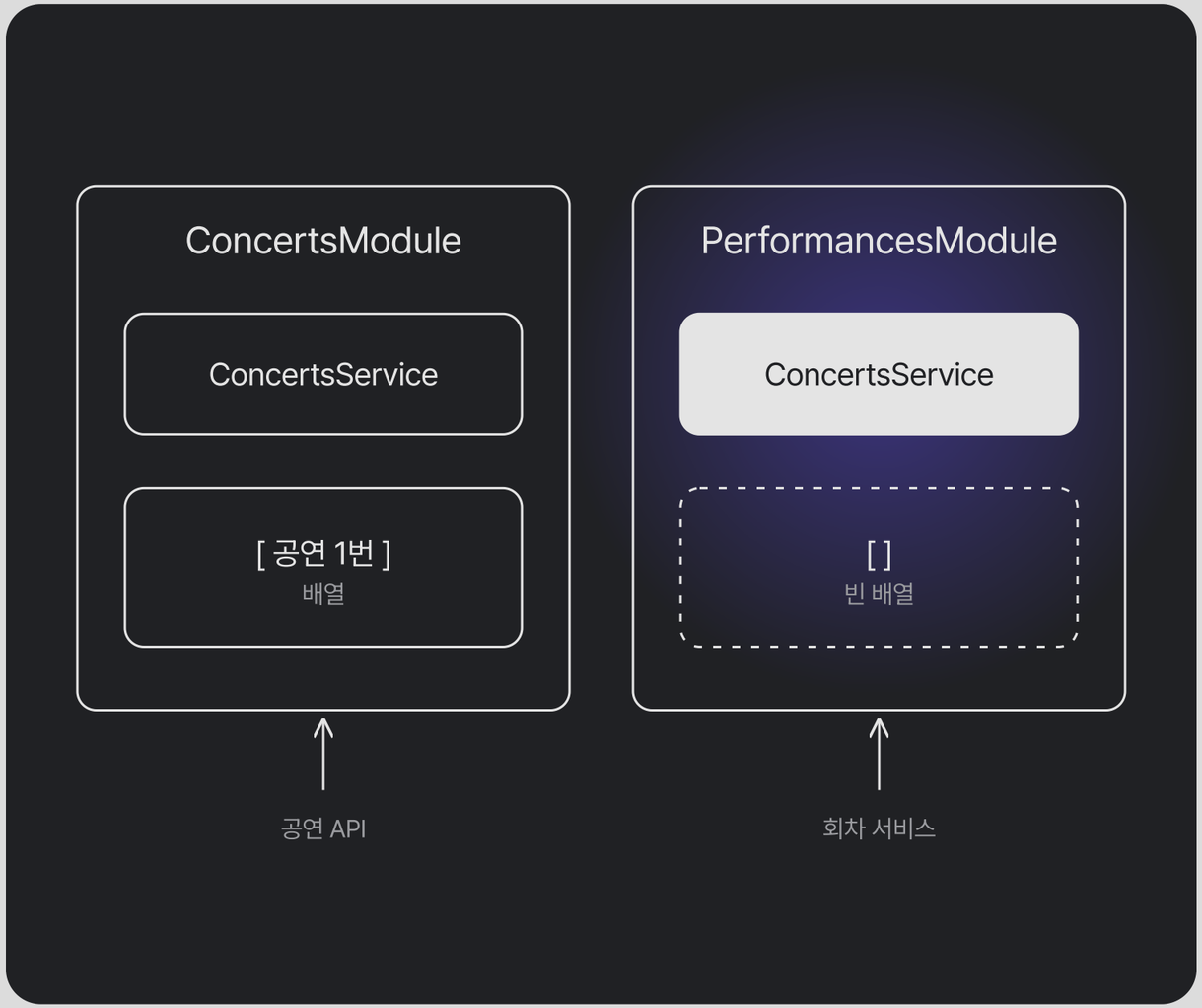
© 한상훈 NestJS 백엔드 마스터

ConcertsService가 두 개

→ ConcertsModule 안의 ConcertsService: 공연 API가 사용, 공연 1번 있음

→ PerformancesModule 안의 ConcertsService: 회차 서비스가 사용, 빈 배열

→ providers 칸: "이 모듈이 직접 만든다"



© 한상훈 NestJS 백엔드 마스터

증상이 늦게 보이는 경우

→ 부팅 성공, 공연 API 정상, 회차 API 정상

→ 어긋남: 서비스 안에 값을 가진 경우에만 표시

→ 예: 서비스 안의 캐시, 카운터, 연결 객체

→ 예: 공연 정보 60초 캐시 → 이름 수정 뒤 한쪽 인스턴스만 새 이름

→ 데이터를 DB에만 두는 서비스: 증상이 바로 안 보임

© 한상훈 NestJS 백엔드 마스터

imports만 넣으면

performances.module.ts

```
@Module({
  imports: [ConcertsModule],
  controllers: [PerformancesController],
  providers: [PerformancesService],
})
export class PerformancesModule {}
```

부팅 로그

```
ConcertsService 생성
ERROR [ExceptionHandler] UnknownDependenciesException [Error]:
Nest can't resolve dependencies of the PerformancesService (?).
```

→ ConcertsModule: 아직 아무것도 내주지 않음

© 한상훈 NestJS 백엔드 마스터

ConcertsModule의 exports

concerts.module.ts

```
 @Module({
   controllers: [ConcertsController, HoldsController, FilesController],
   providers: [ConcertsService],
+  exports: [ConcertsService],
 })
 export class ConcertsModule {}
```

→ exports에는 자기 providers에 있는 것만

→ 다른 모듈을 통째로 다시 내보내기(모듈 re-export): 가능

→ 남의 서비스를 exports에: 부팅 에러 `UnknownExportException` ("Nest cannot export a provider/module that is not a part of the currently processed module")

© 한상훈 NestJS 백엔드 마스터

하나로 돌아온 ConcertsService

○ ○ ○ 터미널

```
ConcertsService 생성
LOG [NestApplication] Nest application successfully started

$ curl -X POST localhost:4013/concerts -d '{"title":"가을 콘서트"}'
{"id":1,"title":"가을 콘서트"}
$ curl -X POST localhost:4013/performances -d '{"concertId":1,"startsAt":"2026-11-14T19:00"}'
{"id":1,"concertId":1,"startsAt":"2026-11-14T19:00"}
$ curl -X POST localhost:4013/performances -d '{"concertId":9,"startsAt":"2026-11-14T19:00"}'
{"type":"about:blank","title":"Not Found","status":404,"detail":"공연 9번이 없습니다","instance":"/performances"}
```

→ 생성 로그 한 번: 공연 API와 회차 서비스가 같은 인스턴스

© 한상훈 NestJS 백엔드 마스터

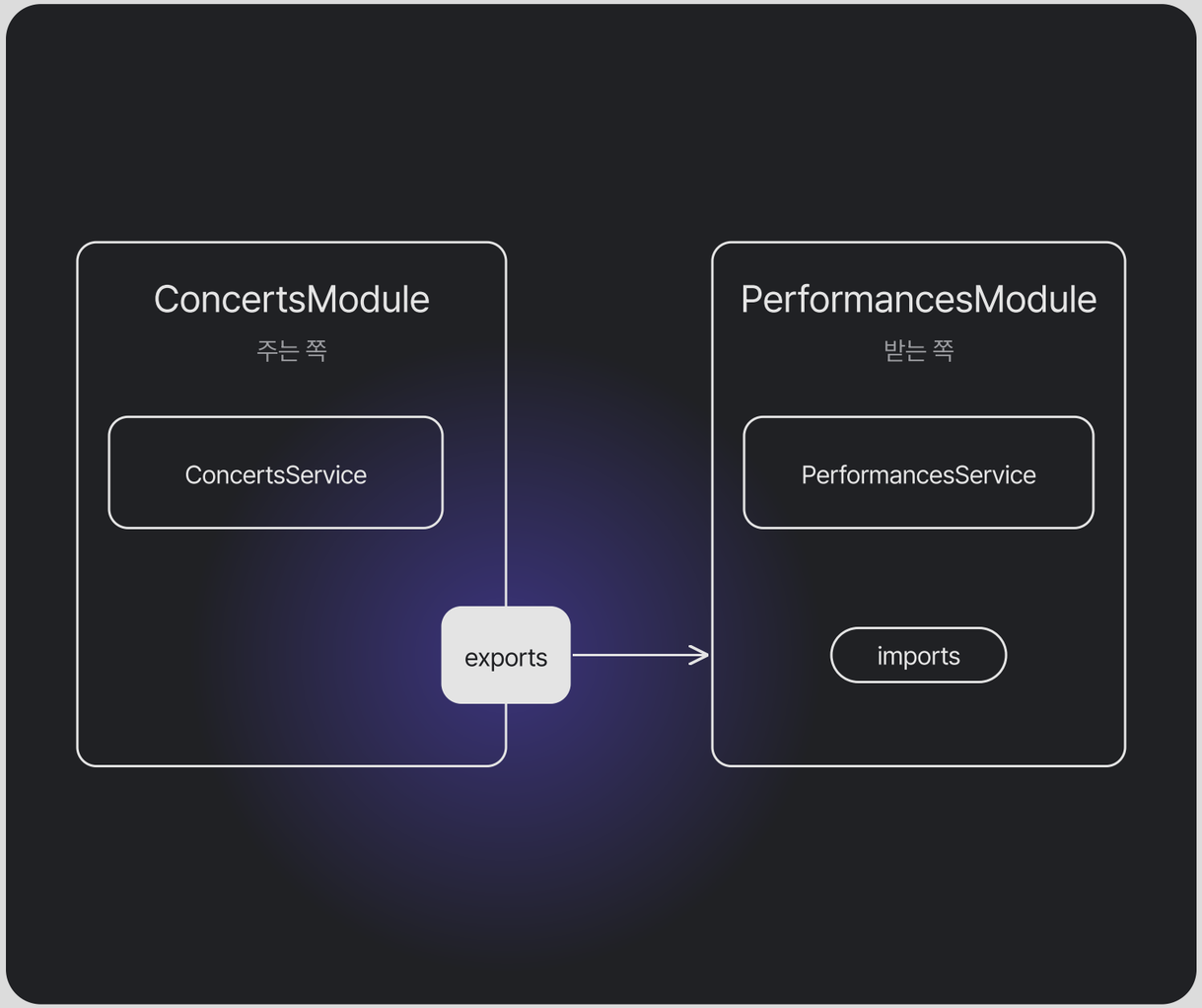
주는 쪽과 받는 쪽

→ 주는 쪽(ConcertsModule): exports에 ConcertsService

→ 받는 쪽(PerformancesModule): imports에 ConcertsModule

→ 둘 중 하나라도 빠지면: 같은 부팅 에러

→ imports에 넣는 것: 서비스 클래스 대신 모듈



© 한상훈 NestJS 백엔드 마스터

열지 않은 프로바이더

→ 예: ConcertsModule 안의 공연 제목 정리 클래스, providers에만 등록

→ 회차 쪽에서 주입 시도: 같은 부팅 에러

→ 모듈 안 클래스: 이름을 바꾸든 둘로 쪼개든 밖은 무관

→ 밖과의 약속: exports에 올린 것만

© 한상훈 NestJS 백엔드 마스터

전부 AppModule 하나에

항목	AppModule 하나에 등록	기능별 모듈
주입 가능 범위	모든 클래스가 모든 서비스	exports로 연 것만
좌석 상태 변경 코드	아무 데서나	좌석 모듈 안에서만
고칠 때 확인할 곳	서버 전체	그 모듈과 exports 사용처

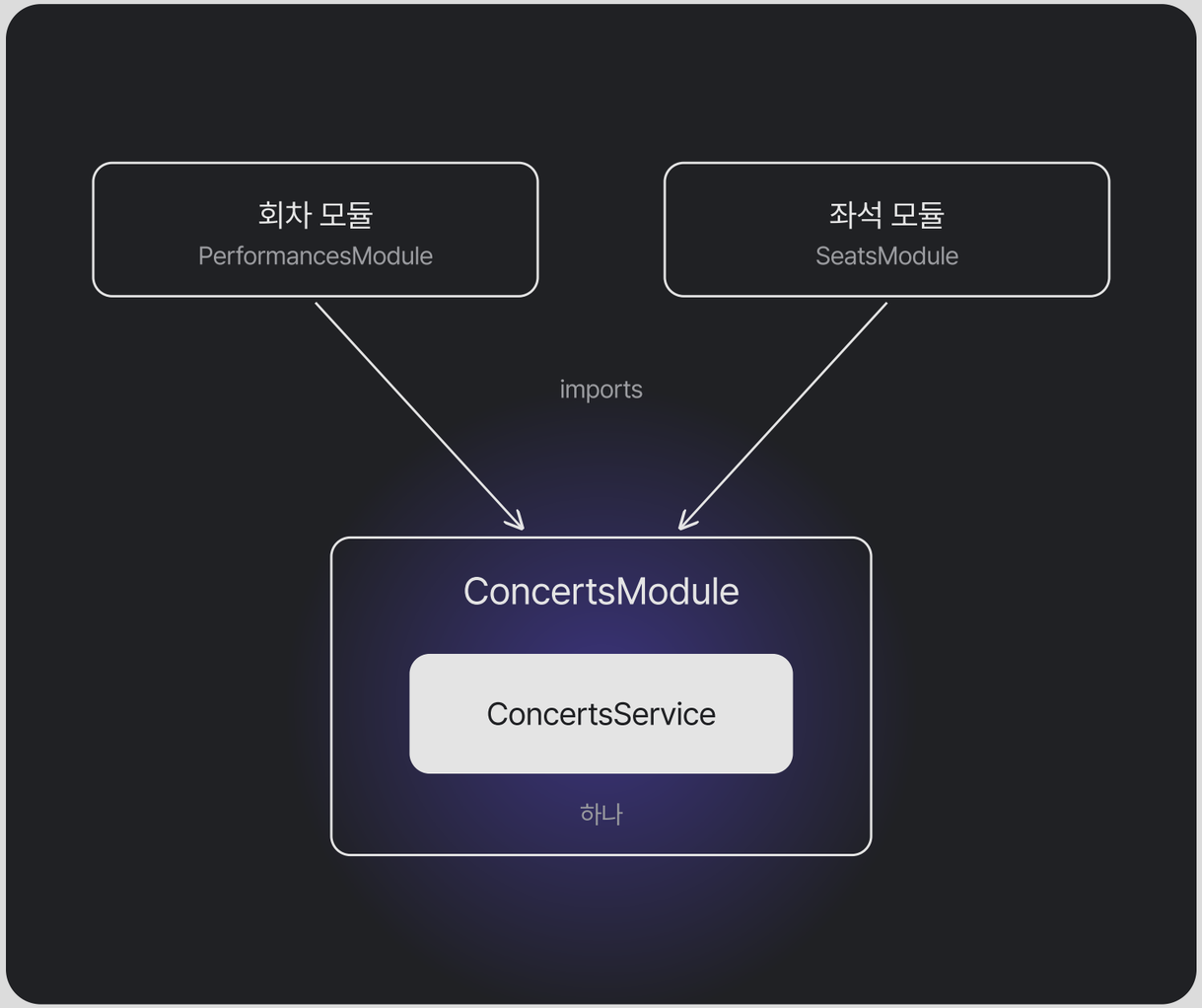
© 한상훈 NestJS 백엔드 마스터

여러 모듈이 가져가도 하나

→ 회차 모듈, 좌석 모듈이 둘 다 ConcertsModule을 imports:
ConcertsService는 하나

→ 모듈도 서버 안에 하나: 두 번 가져가도 같은 모듈

→ 하나가 더 생기는 경우: 다른 모듈 providers에 클래스를 다시 적었을 때



© 한상훈 NestJS 백엔드 마스터

서로를 imports하면

```
// ConcertsModule imports: [PerformancesModule]
// PerformancesModule imports: [ConcertsModule]
$ nest build
Rspack 2.2.9 compiled successfully in 30 ms
$ node dist/apps/ticketing/main.js
ReferenceError: Cannot access 'ConcertsModule' before initialization
```

→ 빌드 통과, 부팅 실패

→ 두 파일이 서로를 import: 한쪽이 만들어지기 전에 다른 쪽이 읽음

→ 의존 방향: 한 방향으로

© 한상훈 NestJS 백엔드 마스터

libs/common의 CommonModule

common.module.ts

```ts
@Module({
  providers: [CommonService],
  exports: [CommonService],
})
export class CommonModule {}
```

→ 라이브러리의 모듈도 같은 규칙: exports로 연 프로바이더만

→ `seatLabel`: 일반 함수, 모듈 규칙과 무관 (import로 바로 사용)

→ 모듈이 관리하는 대상: Nest가 만들어 주입하는 프로바이더

→ 상태 없는 작은 함수: 함수로 / 상태·다른 서비스 필요: 프로바이더로

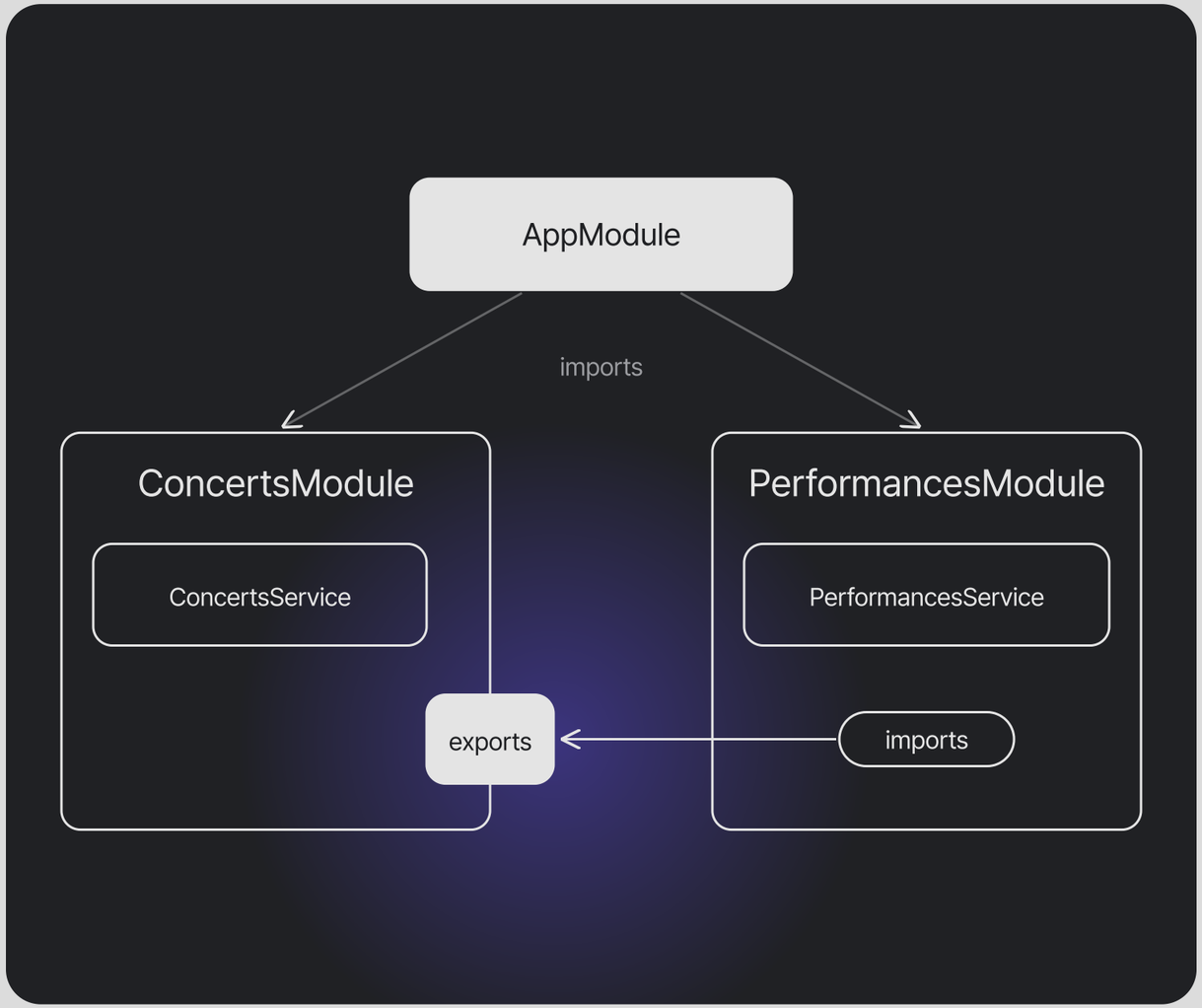
© 한상훈 NestJS 백엔드 마스터

지금의 모듈 그래프

→ AppModule
imports: ConcertsModule, PerformancesModule

→ ConcertsModule: providers ConcertsService, exports ConcertsService

→ PerformancesModule: imports ConcertsModule, providers PerformancesService



© 한상훈 NestJS 백엔드 마스터

부팅 에러를 만나면

확인	빠졌을 때
쓰려는 서비스가 있는 모듈	모듈 그래프에 없음: 로그에 InstanceLoader 줄 없음
그 모듈의 exports	`can't resolve dependencies`
내 모듈의 imports	`can't resolve dependencies`
다른 모듈 providers에 다시 적기	금지: 인스턴스 둘

© 한상훈 NestJS 백엔드 마스터

새 기능은 모듈부터

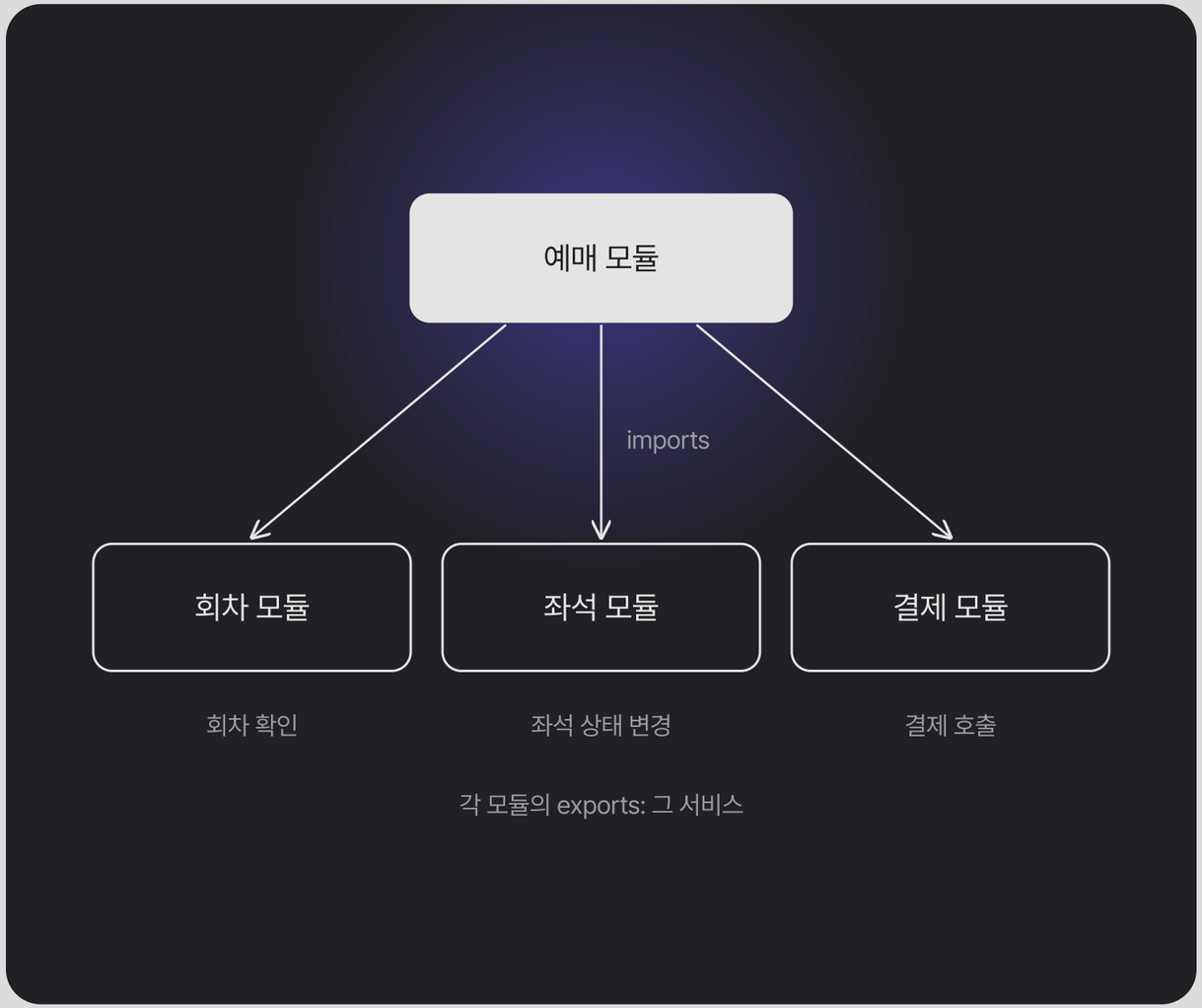
→ 예: 예매 모듈

→ 쓸 것: 회차 확인, 좌석 상태 변경, 결제 호출

→ imports: 회차, 좌석, 결제 모듈

→ 각 모듈의 exports: 그 서비스

→ 코드 작성 전에 정하기: 부팅 에러로 하나씩 고치는 일 방지



© 한상훈 NestJS 백엔드 마스터

모듈의 규칙

"프로바이더는 자기 모듈 안에서만 보이고, 밖에서 쓰려면 exports로 열어 줘야 됩니다."